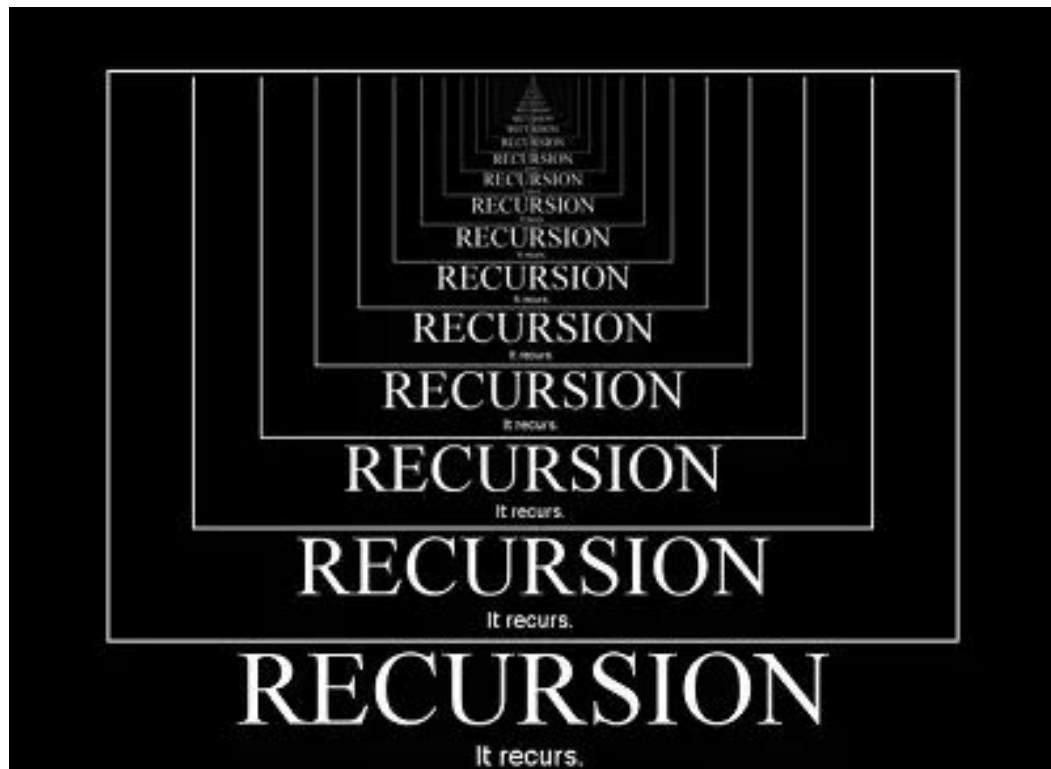

Data Structures

Recursion (Basics)

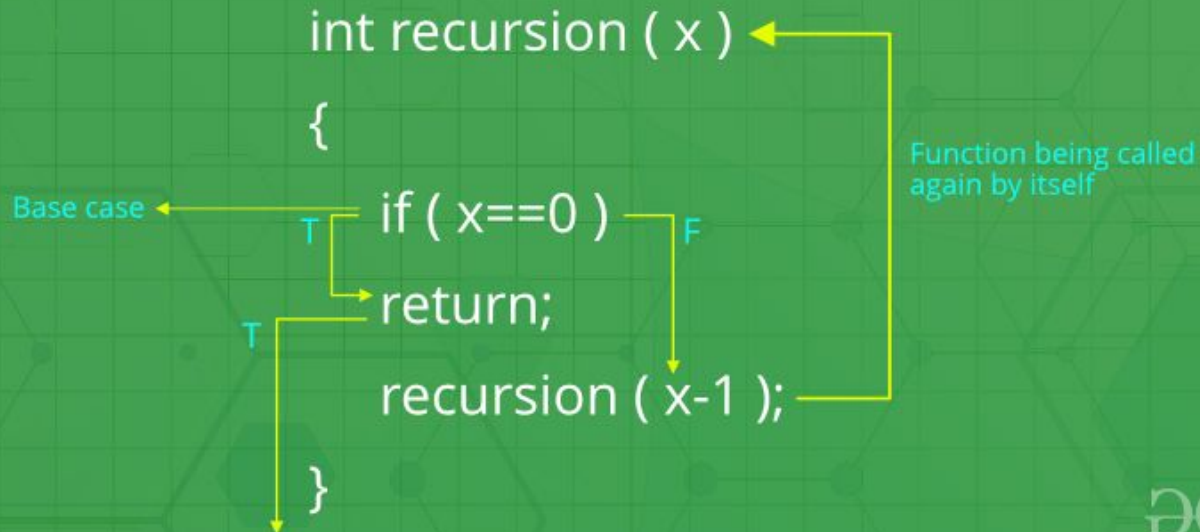
Slides Adapted from
Prantik Paul [PKP]

Recursion



Recursive Definition

Recursive Functions



Recursive Definition (Factorial)

$$1! = 1$$

$$2! = 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$4! = 4 \times 3 \times 2 \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

...

and so on.

Recursive Definition (Factorial)

$$n! = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ n \times (n - 1)! & \text{if } n > 1 \end{cases}$$

Recursive Definition (Factorial)

$$5! = 4 \times 4!$$

$$4 \times 3!$$

$$3 \times 2!$$

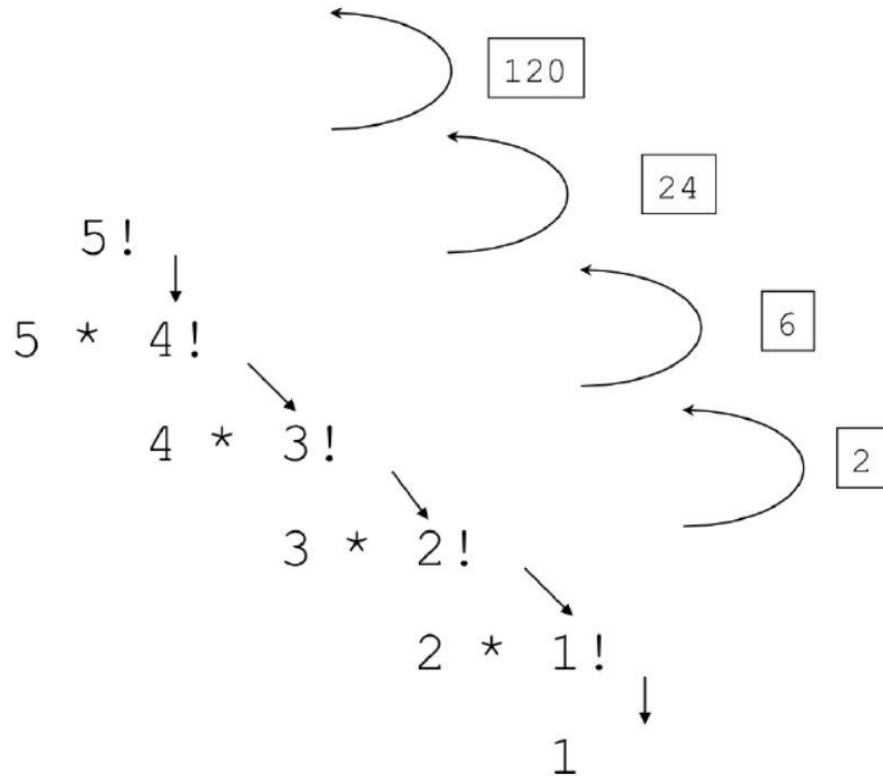
$$2 \times 1!$$

$$1$$

Recursive Programming (Factorial)

```
def fact(n):  
    if n==0 or n==1:  
        return 1    #Base Case  
    else:  
        return n * fact(n-1)    #recursive part
```

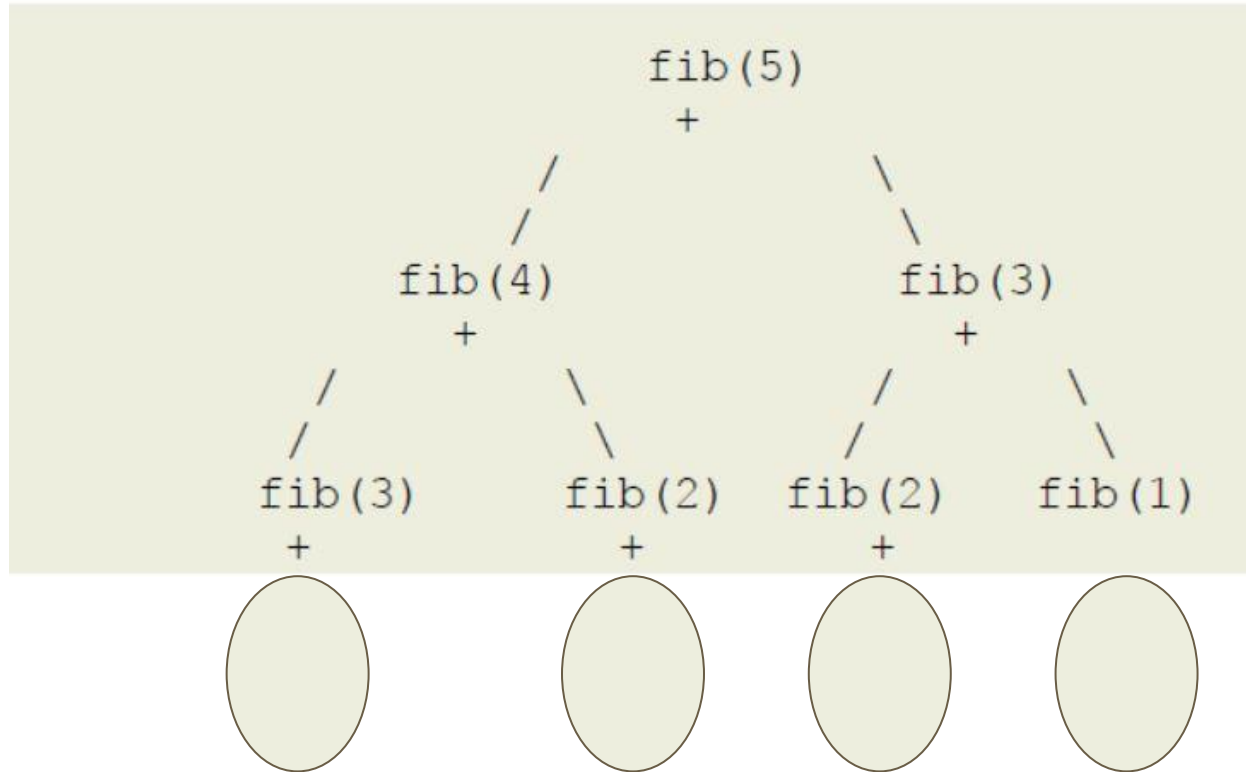
Recursion Tree (Factorial)



Recursive Definition (Fibonacci)

```
fib(n) =  $\begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & n \geq 2 \end{cases}$ 
```

Recursion Tree (Fibonacci)



Sum of numbers

```
def iterativeSum(head):  
    sum=0  
    temp=head  
    while temp!=None:  
        sum+=temp.element  
        temp=temp.next  
    return sum
```

Recursive Definition (Sum of numbers)

```
sum = 
$$\begin{array}{ll} - & \\ | & k \quad \quad \quad k \text{ is the only element} \\ | & \\ | & k + \text{sum}(n.\text{next}) \quad \text{otherwise} \\ - & \end{array}$$

```

Recursive Programming (Sum of numbers)

```
def recursiveSum(head):  
    if head.next==None:  
        return head.element    #Base case  
    else:  
        return head.element+recursiveSum(head.next)
```

Recursive Definition (Length of String)

```
len(s) =  $\begin{cases} 0 & \text{if string s is empty} \\ 1 + \text{len(rest)} & \text{otherwise} \end{cases}$ 
```

Recursive Programming (Length of String)

```
def strLength(str):  
    if len(str)==0:  
        return 0          #Base case  
    else:  
        return 1+strLength(str[1: ])
```

Recursive Definition (Length of Linked List)

```
len(l) =  $\begin{cases} 0 & \text{if } l \text{ is an empty list} \\ 1 + \text{len}(\text{rest}) & \text{otherwise} \end{cases}$ 
```

Recursive Programming (Length of Linked List)

```
def listLength(head):  
    if head==None:    #empty linked list  
        return 0      #base case  
    else:  
        return 1+listLength(head.next)    #Recursive part
```

Recursive Definition (Sequential Search LL)

```
contains(l,k) =  $\begin{cases} \text{false} & \text{if list l is empty} \\ \text{true} & \text{if l.item = k} \\ \text{contains(l.next,k)} & \text{otherwise} \end{cases}$ 
```

Recursive Programming (Sequential Search LL)

```
def contains(head,key):  
    if head==None:  
        return False    #base case  
    elif head.element==key:  
        return True     #base case  
    else:  
        return contains(head.next,key)    #recursive part
```

Recursive Programming (Sequential Search Array)

```
def contains(arr,key):  
    if len(arr)==0:  
        return False    #base case  
    elif arr[0]==key:  
        return True     #base case  
    else:  
        return contains(arr[1: ],key)    #recursive part
```

Recursive Definition (Seq Search : Left Index)

```
contains(a,l,k) =  $\begin{cases} \text{false} & \text{if } l \geq a.\text{length} \\ \text{true} & \text{if } a[l] = k \\ \text{return contains}(a,l+1,k) & \text{otherwise} \end{cases}$ 
```

Recursive Programming (Seq Search : Left Index)

```
def contains(arr, left, key):  
    if left >= len(arr):  
        return False    #base case  
    elif arr[left] == key:  
        return True     #base case  
    else:  
        return contains(arr, left+1, key)    #recursive part
```

Recursive Programming (Find Maximum - Linear LL)

```
def maximum(a,b):  
    return a if a>=b else b  
  
def findMax(head):  
    if head.next==None:  
        return head.element    #base case  
    else:  
        maxRest=findMax(head.next)    #recursive part  
        return maximum(head.element,maxRest)
```

Recursive Programming (Find Maximum - Linear Array)

```
def maximum(a,b):  
    return a if a>=b else b  
  
def findMax(arr, left):  
    if left == len(arr)-1:  
        return arr[left]    #base case  
    else:  
        maxRest=findMax(arr, left+1)    #recursive part  
        return maximum(arr[left], maxRest)
```

Recursive Definition (Exponentiation)

$$a^n = \begin{cases} 1 & n = 0 \\ a \times a^{n-1} & n > 0 \end{cases}$$

Recursive Programming (Exponentiation)

```
def exp(a, n):  
    if n==0:  
        return 1      #base case  
    else:  
        return a * exp(a, n-1)    #recursive part
```